Contents

Mixed Precision Allocation Under Visual Distribution Shift — Project Deep Dive 1

Part 1 — The project in simple English 1
1.1 The thing being studied . . . 1
1.2 What people do today, and why it might be wrong . . . 2
1.3 Why it plausibly goes wrong . . . 2
1.4 The measurement . . . 2
1.5 The fairness rule . . . 3
1.6 The data . . . 3

Part 2 — Deep dive 3
2.1 Architecture and where the weight actually lives . . . 3
2.2 The eight configurations and their memory structure . . . 4
2.3 Why NF4 makes the result cleaner than the earlier design . . . 5
2.4 Selection procedure . . . 5
2.5 Statistics . . . 5
2.6 Threats to validity . . . 6
2.7 Current state and the critical path . . . 7

Part 3 — Minimum specifications 7
3.1 The hard requirement . . . 7
3.2 Minimum viable configuration . . . 7
3.3 Recommended configuration . . . 8
3.4 Disk breakdown . . . 8
3.5 Compute budget . . . 9
3.5.1 If hardware is the constraint . . . 9

Mixed Precision Allocation Under Visual Distribution Shift — Project Deep Dive

Analysis document. Written 2026-10-22 for orientation and planning. This is not the protocol and not a results document. The authoritative specification is PROPOSAL_SNAPSHOT.md (the proposal's Final tab); protocol changes belong in DECISIONS.md; real status lives in STATUS.md.

Numbers in this document are labelled either measured, derived, or estimated. Nothing here is an experimental result. No experiment has been run yet.

Part 1 — The project in simple English

1.1 The thing being studied

A vision-language model (VLM) answers questions about pictures. The one in this study, SmolVLM2-2.2B, has three working parts:

Part	Job	Analogy
Vision encoder	Looks at the image and turns it into numbers	The eye
Connector	Translates those numbers into the language model's vocabulary	The optic nerve

Part	Job	Analogy
Language backbone	Reads the translation and writes the answer	The brain

Quantization makes a model smaller by storing its weights with fewer bits. Normally each weight is a 16-bit number. Store it in 4 bits instead and that part of the model takes about a quarter of the space — but the numbers get rounded, so the model gets slightly worse at its job.

The useful fact: you don't have to compress all three parts equally. You can squeeze the language backbone to 4 bits and leave the eye at full precision, or the other way round. With two choices (4-bit or 16-bit) for each of three parts, there are $2 \times 2 \times 2 = 8$ possible configurations.

1.2 What people do today, and why it might be wrong

If you want to deploy one of these models on a phone or a small device, you have a memory limit. So you try the configurations that fit, test them on some images, and ship whichever scored best.

The images you test on are almost always clean, well-lit, in-focus photos — because those are the images that come with benchmarks.

The images your model actually sees in the field are often not clean. A security camera at night. A phone photo through a dirty lens. A warehouse in bad light.

So: is the configuration you picked using clean photos still the right configuration once the real images are dark or blurry?

Nobody has checked. That's the project.

1.3 Why it plausibly goes wrong

Here is the mechanism that makes this more than a bookkeeping exercise.

Blur destroys fine detail in an image. Quantization adds rounding noise. Both degrade the same signal — the visual information flowing from the eye, through the optic nerve, into the brain. They might compound.

If they do, the best configuration could change depending on image quality:

- On clear photos, the vision encoder has a strong, clean signal to work with, so rounding its weights costs little. The sensible move is to protect the language backbone.
- On blurry photos, the visual signal is already weak. Now rounding the weights that process it might be the thing that breaks the answer — so protecting the vision side could matter more.

Same memory budget, different right answer. If that's true, then everyone tuning compression on clean benchmark images is shipping a misconfigured model.

1.4 The measurement

The whole study reduces to one subtraction. Build a test set of dark and blurry images. Then pick a configuration two different ways:

- Way A (what people do): pick the configuration that scores best on clean images.
- Way B (the ideal): pick the configuration that scores best on dark and blurry images.

Run both on the same dark/blurry test set:

$$\Delta = (\text{accuracy of the config picked way B}) - (\text{accuracy of the config picked way A})$$

- Δ is large and positive → picking on clean images cost you real accuracy. Current practice is wrong, and you've measured by how much.

- Δ is about zero → the choice travels fine. Calibrate on whatever is convenient. This confirms an assumption the field currently makes with no evidence behind it.

Both outcomes are publishable. This is the single best feature of the design. Most research projects fail if the effect isn't there; this one answers its question either way. A null result is a real contribution, provided the confidence intervals are tight enough to rule out an effect that would matter.

There's a third, practical option too: pick the configuration using a mixture — mostly clean images with 25%, 50%, or 75% degraded ones. If a small amount of realistic data recovers most of the loss, that's directly actionable advice for practitioners, who can usually collect a few hundred representative images but not a few hundred thousand.

1.5 The fairness rule

Comparing a 4.5 GB model against a 1.5 GB model proves nothing — of course the bigger one wins. So every comparison happens under a memory cap: only configurations that fit the same budget compete against each other.

This is what turns the question from "does compression hurt" (obviously yes) into "given a fixed budget, where should you spend your bits" — which is the question an engineer actually faces.

1.6 The data

Two datasets, chosen deliberately to look nothing like each other:

Dataset	Content	Question style
POPE-COCO	Everyday scenes and objects	"Is there a dog in this image?"
CelebA	Cropped human faces	"Is this person wearing glasses?"

Both are yes/no, so scoring is unambiguous — no judging free-text answers. And because the two datasets share almost no visual statistics, if the transfer penalty differs between them, that difference is attributable to image content rather than to overlap.

Then each image is used in seven conditions: the original, plus three levels of darkening and three levels of blur. Same image, same question, same correct answer — only the picture quality changes. That pairing is what makes the comparison clean.

Part 2 — Deep dive

2.1 Architecture and where the weight actually lives

Component sizes derived from the published `config.json` for `HuggingFaceTB/SmolVLM2-2.2B-Instruct` (revision `482adb53…`): a SigLIP-SO400M/14 @384 vision tower, a pixel-shuffle MLP connector (`scale_factor=3`), and a SmolLM2-1.7B text backbone.

Component	Params	Share	FP16
Vision encoder (linear layers)	411.1 M	18.3%	822 MB
Vision encoder (patch/pos embeddings)	1.5 M	0.1%	3 MB
Connector	21.2 M	0.9%	42 MB
Text backbone (linear layers)	1,610.6 M	71.7%	3,221 MB
Text embedding	100.9 M	4.5%	202 MB
`lm_head`	100.9 M	4.5%	202 MB

Component	Params	Share	FP16
Total	2,246.3 M	100%	4,493 MB

Cross-check: this derivation totals 4.493 GB in FP16 against the measured 4,493,569,888 bytes recorded in M1_MAC_FINDINGS.md — agreement to within 0.02%. The breakdown can be trusted for planning, though M1 must confirm it module by module.

The connector problem

The connector is 0.9% of the weights. This is the most important structural fact in the project and it has consequences the proposal does not yet address.

Quantizing the connector saves roughly 30 MB — 0.7% of the model. So in any accuracy-versus-memory plot, the connector's precision is nearly free. That cuts both ways:

- Bad news: the connector cannot be a meaningful lever for hitting a memory cap. You will never need to quantize it to fit a budget.
- Good news: it makes the central hypothesis cheap to test. If blur damage and quantization noise really do compound at the connector, you get that finding for 30 MB, which is an unusually clean result — a precision decision with negligible memory cost and a measurable accuracy consequence.
- Design consequence: if connector precision shows any accuracy benefit under shift, the practical recommendation is simply "always keep the connector at FP16." That is a useful, concrete deployment rule, but note it is a conclusion about accuracy only, not about the memory trade-off.

2.2 The eight configurations and their memory structure

Estimated weight memory per configuration, pending the M1 audit. Assumes NF4 at 0.5 bytes/param plus one FP32 `absmax` per 64-value block (0.5625 bytes/param effective, double quantization off), and that embeddings, norms, and `lm_head` are excluded from conversion — the standard bitsandbytes default.

Vision	Connector	Backbone	Weight MB	% of FP16
4-bit	4-bit	4-bit	1,556	35%
4-bit	FP16	4-bit	1,586	35%
FP16	4-bit	4-bit	2,147	48%
FP16	FP16	4-bit	2,177	48%
4-bit	4-bit	FP16	3,871	86%
4-bit	FP16	FP16	3,902	87%
FP16	4-bit	FP16	4,462	99%
FP16	FP16	FP16	4,493	100%

The structure that matters: the eight configurations form four tight pairs. Within each pair the only difference is the connector (~30 MB apart). Between pairs, the gaps are large — ~560 MB for the vision encoder, ~1,700 MB for the backbone.

This is good news for the protocol's requirement of "at least two feasible configurations at a common cap." A cap just above 2,177 MB admits four configurations that differ in vision and connector precision; a cap near 3,902 MB admits six. The requirement is comfortably satisfiable.

But it reframes what the study is really measuring. Because the backbone dominates memory so heavily, the interesting within-cap comparisons are almost entirely vision-encoder and connector decisions made at a fixed backbone precision. That is well matched to the research question — the hypothesis is about the visual pathway — but the write-up should state it plainly rather than implying a free three-way trade-off.

One risk to check during M1: whether the four pairs are effectively duplicates in accuracy as well as memory. If connector precision turns out to make no measurable accuracy difference, the design collapses from 8 configurations to 4, which halves the inference budget — a welcome saving, but it must be established empirically, not assumed.

2.3 Why NF4 makes the result cleaner than the earlier design

An earlier version of this proposal used GPTQ or AWQ, which are calibration-dependent: they examine sample data and adjust the quantized weights accordingly. The final design uses bitsandbytes NF4, which is data-free — the quantized weights are a deterministic function of the original weights alone.

This is a genuine improvement in experimental hygiene, not just a simplification. With a calibration-dependent quantizer, a positive Δ has two possible causes that are hard to separate: the configuration choice was wrong, or the calibration data was unrepresentative. With NF4 there is no calibration step, so a positive Δ can only come from the configuration choice. The confound is designed out rather than controlled for.

The cost is that the study no longer speaks to calibration-set selection, which is what most of the related literature is about. The related-work framing should therefore cite that literature as motivation — methods like MODE, SPEED-Q, and MASQuant all measure component sensitivity on a sample and allocate bits from that measurement, and none test whether the measurement transfers — while being explicit that this study tests a cleaner, narrower version of the question.

2.4 Selection procedure

At each memory cap with at least two feasible configurations:

1. Score every feasible configuration on the development set.
2. Do this three ways: clean-only, target-only, and mixed (25%, 50%, 75% target).
3. Take the argmax for each. Break ties toward lower memory, then by a fixed predetermined order.
4. Evaluate the selected configurations on the untouched target test set.
5. Δ = target-selected accuracy − source-selected accuracy, in percentage points.

Four invariants make this valid, and each deserves a test:

- Equal budgets. Every selection strategy sees the same number of development examples and the same number of distinct images. Otherwise a positive Δ could just mean “more data wins.”
- Common candidate set. All strategies choose from the same feasible configurations at a given cap.
- Fixed everything else. Prompts, image resolution, decoding settings, and answer parsing are frozen before selection begins.
- Test set never consulted. Both Δ terms come from development data. The selector must be structurally incapable of reading test accuracy — enforce it in code, not by discipline.

2.5 Statistics

Paired cluster bootstrap, resampling by original image (by identity for CelebA), keeping all questions and all seven corrupted variants of an image together in whatever resample it lands in.

Why clustering is mandatory: the seven conditions of one image are not seven independent observations. They share a subject, a scene, and a correct answer. Treating them as independent would shrink the confidence intervals by roughly the square root of the cluster size and manufacture significance that isn’t there. The same applies to CelebA identities, where one person appears in many images.

Why pairing matters: both Δ terms are measured on the same test items, so the comparison is within-item. Bootstrapping the difference directly is far more powerful than comparing two independently-bootstrapped accuracies, because the item-difficulty variance cancels.

Selection stability is a separate analysis from Δ. Resample the development set and re-run selection; record how often each configuration wins. If the winner flips constantly across resamples, then Δ is measuring noise

in the selection procedure rather than a real property of the model — and that finding must be reported, not buried.

Reporting a null honestly means stating which effect sizes the study can exclude. “No significant difference” with an interval of ±8 points says almost nothing. The same statement with ±1.5 points is a real invariance result. This distinction decides whether a null outcome is a contribution or a failed experiment, so power should be sanity-checked at the pilot stage, not discovered at the end.

2.6 Threats to validity

#	Threat	Why it bites	Mitigation
1	Component control doesn’t actually work	bitsandbytes may convert modules you didn’t target, or silently skip ones you did. A config accepted by the loader is not a config that exists.	Per-module dtype and quantization-state dump for all 8 configs. This is M1’s acceptance gate. A hard blocker if it fails — report it, don’t work around it.
2	Severities miss the useful window	Too mild and nothing changes; too severe and every config fails equally. Either way $\Delta \approx 0$ for an uninteresting reason.	The 100-question pilot must show graded degradation — accuracy declining with severity but not collapsing to chance. Pick severities on pilot data only.
3	Split leakage	Splitting after corruption, or by question instead of by image, puts variants of one image on both sides. Results become meaningless and the error is nearly invisible.	Split by original image before generating variants; split CelebA by identity. Automated leakage test with synthetic fixtures.
4	Configurations are effectively duplicates	If connector precision changes nothing, 8 configs are really 4.	Detect during M1 via effective-signature deduplication; reduces cost if confirmed.
5	Selection noise swamps the effect	With 8 candidates and a small dev set, the argmax is unstable; Δ then measures variance.	Report selection frequencies alongside Δ. Size the dev set from pilot variance.
6	Invalid answers smuggle in bias	A model emitting unparseable text looks “wrong” in a way that may correlate with configuration — so parsing rules could drive the result.	Count invalid as incorrect, fix the parser before selection, report invalid rate per configuration, and keep infrastructure failures distinguishable from model answers.
7	8 GB ceiling	The FP16 reference is the largest configuration; OOM at high image resolution is plausible.	Batch size 1, fixed modest resolution, pilot before committing. Documented fallback to SmolVLM2-500M.

#	Threat	Why it bites	Mitigation
8	Schedule	The proposal froze the protocol on Oct 12; it is now Oct 22 and M1 is blocked on hardware access.	The critical path is a CUDA machine, not code volume. See §2.7.

2.7 Current state and the critical path

From STATUS.md and the `Kaushal-Commits` branch:

Milestone	Status
M0 — environment, model revision pinned	Done on macOS; not on CUDA
M1 — component control + audit	FP16 audit implemented and CPU-load-tested; NF4 blocked, inference never validated
M2 — data and corruption pipeline	Not started
M3 — pilot, prediction cache, protocol freeze	Not started
M4 — selection and statistics	Not started
M5 — main runs and report	Not started

What exists: `src/mpvs/{model,quantization,memory,cli}.py`, 7 passing unit tests, a pinned macOS environment (PyTorch 2.14.1, Transformers 4.52.4 — 5.20.1 was rejected after its SmolVLM image-token merger failed), the model pinned to revision `482adb537c021c86670beed01cd58990d01e72e4`, and the measured FP16 storage figure used above.

The single blocker is NF4 on CUDA. bitsandbytes 4-bit requires an NVIDIA GPU; Apple Silicon cannot run it, and the M1 host additionally reported `mps_available=False`. Everything downstream of M1 — the data pipeline, the pilot, the cache, the statistics — is independent of this blocker and could proceed in parallel on any machine.

Recommended sequencing. Do not wait on GPU access to start M2. The corruption pipeline, the split logic, the leakage tests, and the answer parser need no GPU and are on the critical path to the pilot. Getting onto CUDA hardware unblocks M1; writing M2 in the meantime is free progress.

Part 3 — Minimum specifications

3.1 The hard requirement

An NVIDIA GPU with CUDA. This is not a preference. bitsandbytes NF4 — the quantizer the entire study is built on — has no Apple Silicon or CPU backend. Without it there is no 4-bit configuration, and with no 4-bit configuration there is no experiment.

Apple Silicon is usable for development only: writing code, running unit tests, auditing module names, and loading FP16 on CPU. That is exactly how far the existing M1 work got, and it is the correct use of that hardware.

3.2 Minimum viable configuration

Resource	Minimum	Reasoning
GPU	NVIDIA, 8 GB VRAM, compute capability ≥ 7.5 (Turing: RTX 2060 / T4 or newer)	The FP16 reference is the largest configuration at ~4.5 GB of weights; the rest is activations and fragmentation headroom. Verify the CC floor against your installed bitsandbytes version.
System RAM	16 GB	The checkpoint is loaded and inspected on CPU before transfer. Use `low_cpu_mem_usage=True` to keep peak near model size.
Disk	25 GB free	See §3.4 breakdown.
CPU	Any modern 4-core	Not a bottleneck; image decoding and corruption are cheap.
CUDA	11.8 or 12.x, driver matched to the PyTorch build	Must match the bitsandbytes wheel.
Python	3.11	Matches the pinned `mpvs-m1` environment.
OS	Linux strongly preferred	bitsandbytes on Windows is viable but less well-tested.

3.3 Recommended configuration

Resource	Recommended	Benefit
GPU	16–24 GB (RTX 4080/4090, A4000/A5000, L4, A100)	Removes OOM risk entirely, allows batching above 1, cuts wall-clock several-fold.
System RAM	32 GB	Comfortable headroom for the FP16 load path.
Disk	50 GB SSD	Room for both datasets uncropped, cached corrupted variants, and all prediction caches.

An 8 GB card is sufficient for the core study as specified — one configuration in memory at a time, batch size 1 — and the proposal is correctly scoped to it. The upgrade buys time, not feasibility.

3.4 Disk breakdown

Item	Size	Notes
SmolVLM2-2.2B checkpoint (HF cache)	~4.5 GB	FP16 safetensors
COCO val2014 images	~6.3 GB	Or ~80 MB if you fetch only the ~500 POPE-referenced images
CelebA aligned images	~1.4 GB	Must stay internal under the research-use agreement
Corrupted variants	0–2 GB	Zero if generated deterministically on the fly — the better choice
Prediction cache	~100 MB	~90k JSON records

Item	Size	Notes
Conda environment	~6 GB	PyTorch and CUDA libraries dominate
Total	~19–21 GB	25 GB minimum, 50 GB comfortable

3.5 Compute budget

Derived answer counts from the proposal's design:

Run	Count	Arithmetic
Test	56,000	500 questions × 7 conditions × 8 configs × 2 domains
Development (illustrative at 300 q/domain)	33,600	300 × 7 × 8 × 2
Pilot	~5,600	100 × 7 × 8
Total	~95,000	

Estimated wall-clock, to be replaced by real pilot measurements: at batch size 1 with yes/no outputs of a few tokens, expect roughly 0.3–1.0 s per answer on an 8 GB card. NF4 is often slower per token than FP16 despite being smaller, because weights are dequantized on the fly — do not assume the compressed configurations run faster. That puts the full sweep at roughly 15–40 GPU-hours, plus per-configuration model loading overhead.

This is comfortably feasible as an overnight-and-weekend job. Measuring it is the pilot's main job — treat these numbers as planning figures with no evidential weight.

Two multipliers worth protecting:

- Caching is what makes this tractable. Each (config, condition, question) triple is inferred once and stored. Every memory cap, every mixture ratio, and every bootstrap resample is then recomputed from the cache with no GPU involvement. Without caching, each new analysis means re-running inference.
- Cache keys must be complete — model and processor revision, actual audited quantization state, image transformation, prompt, and decoding settings. An incomplete key silently serves one configuration's answers for another, which is the kind of bug that invalidates a whole study without ever raising an error.

3.5.1 If hardware is the constraint

Documented fallbacks, in the order they cost the least scientifically:

1. Reduce severity count from 3 to 2 per corruption → 5 conditions instead of 7, a 29% cut.
2. Drop to SmolVLM2-500M. Same architecture family, so the three-component structure is preserved; roughly 1 GB at FP16, runs anywhere. Conclusions narrow to the smaller model.
3. Rent rather than buy. The full sweep is tens of GPU-hours, which at current spot rates for a 24 GB card is a very small amount of money and removes the blocker this week rather than whenever hardware appears.

Option 3 deserves serious consideration given that the protocol-freeze date has already passed and the project is otherwise blocked on hardware access.